UNIVERSIDAD POLITÉCNICA DE VICTORIA

IMPLEMENTACIÓN DE FLUENTES DE ESTADO MULTIVALUADOS EN MDP-PROBLOG MEDIANTE DISYUNCIONES ANOTADAS.

T E S I N A

QUE PARA OBTENER EL GRADO DE

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

PRESENTA:

ÁNGEL IVÁN CABRERA ROJAS

DIRECTOR

DR. SAID POLANCO MARTAGÓN

CO-DIRECTOR

DR. HÉCTOR HUGO AVILÉS-ARRIAGA

ORGANISMO RECEPTOR

UNIVERSIDAD POLITÉCNICA DE VICTORIA

CIUDAD VICTORIA, TAMAULIPAS, FEBRERO DE 2026

Ciudad Victoria, Tamaulipas, a **25 de Enero de 2026**

UNIVERSIDAD POLITÉCNICA DE VICTORIA
DR. HÉCTOR HUGO AVILÉS ARRIAGA
PRESENTE

La Universidad Politécnica de Victoria tiene a bien presentar a **CABRERA ROJAS ANGEL IVAN** estudiante del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN**, con número de matrícula **2230343** y seguro facultativo IMSS número **86-18-04-3681-7;** quién deberá realizar su práctica profesional de **ESTADÍA**, a partir del **07 de Enero de 2026** al **17 de Abril de 2026**, con duración de **600 horas** desarrollando el proyecto **"Implementación de disyunciones anotadas en MDP-ProbLog"** que le fué asignado.

Al concluir se le extenderá la carta de liberación al evaluarlo satisfactoriamente y además le solicitaremos su valiosa opinión respondiendo el formulario que se le enviará por mail, referente al desempeño del practicante, la información que nos proporcione, es de vital importancia para la mejora de los programas académicos que ofrece la Universidad Politécnica de Victoria para la formación de profesionistas altamente especializados.

El practicante deberá cumplir con el Reglamento Interno aplicable al personal en su centro de trabajo.

Sin otro particular.

ATENTAMENTE

OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN

C.C.P. SAIDPOLANCOMARTAGÓN
ASESOR INSTITUCIONAL

UNIVERSIDAD POLITÉCNICA DE VICTORIA

Victoria Tamaulipas, a 25 de Enero del 2026
Asunto: Carta de Aceptación

M.A OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN
UNIVERSIDAD POLITÉCNICA DE VICTORIA
PRESENTE

Hacemos de su conocimiento que hemos aceptado al estudiante **CABRERA ROJAS ANGEL IVAN** del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN**, con número de matrícula **2230343** de la Universidad Politécnica de Victoria, para realizar su **ESTADÍA** en nuestra empresa **UNIVERSIDAD POLITÉCNICA DE VICTORIA**, durante el periodo comprendido del día **07 de Enero de 2026** al **17 de Abril de 2026**, el estudiante estará colaborando en el proyecto **"Implementación de disyunciones anotadas en MDP-ProbLog"** con una carga horaria total de **600 horas**

Al concluir satisfactoriamente sus prácticas profesionales, se le entregará al estudiante su carta de liberación debidamente formalizada.

Con el objetivo de colaborar en la mejora de los programas académicos de la Universidad Politécnica de Victoria, estamos de acuerdo en responder a la brevedad posible el formulario de evaluación del desempeño del practicante previo a emitir la liberación de **ESTADÍA**.

Sin otro particular.

ATENTAMENTE
DR. HÉCTOR HUGO AVILÉS ARRIAGA
ASESOR EMPRESARIAL

C.C.P. ARCHIVO

Ing. Juan Camaney de Alba Rojas
PRESENTE

INSERTAR AQUÍ
DOCUMENDO
DE CARTA
DE LIBERACION
DEBIDAMENTE
FIRMADO

CARTA DE ACEPTACIÓN DEL DOCUMENTO PARA SU IMPRESIÓN

Cd. Victoria, Tamaulipas a x

Ángel Iván Cabrera Rojas
PRESENTE

Le comunico que el Programa Académico de Ingeniería en Tecnologías de la Información le ha otorgado la autorización para la impresión de su Tesina de Estadía Práctica cuyo título es:

Implementación de fluentes de estado multivaluados en MDP-ProbLog mediante disyunciones anotadas.

ATENTAMENTE

Dr. Said Polanco Martagón
ASESOR INSTITUCIONAL

c.c.p Director de programa académico

EVALUACIÓN DE ESTADÍA
Rúbrica para evaluación de la presentación y el reporte de estadía

Nombre del alumno: **ÁNGEL IVÁN CABRERA ROJAS** Calificación final: _____

Periodo: **X**

Ponderación	Aspecto a Evaluar	Competente 10	Independiente 9	Básico Avanzado 8	No Competente 5
40	Resultados y Actividades	Estrechamente relacionados al perfil de egreso de su programa académico	Parcialmente relacionados al perfil de egreso de su programa académico	Escasamente relacionados al perfil de egreso de su programa académico	Escasamente relacionados al perfil de egreso de su programa académico
30	Exposición de las actividades de la estadía	Detalladas y sustentadas con respecto a los resultados que se obtuvieron	Detalladas y sustentadas parcialmente con respecto a los resultados que se obtuvieron	Detalladas parcialmente con respecto a los resultados que se obtuvieron	Detalladas escazamente con respecto a los resultados que se obtuvieron
10	Material visual Lenguaje verbal	Uso el lenguaje y la terminología apropiadas; El material visual está organizado, adecuado y suficiente	Uso el lenguaje y la terminología apropiadas El material visual está parcialmente organizado y es suficiente	Uso el lenguaje y la terminología son parcialmente apropiadas; El material visual está parcialmente organizado y es suficiente	Uso el lenguaje y terminología es inapropiado; El material visual no está organizado y es insuficiente
10	Exposición en Idioma Inglés	Pronunciation is clear so language is easily understood (2.5) Uses fluent connected speech, occasionally disrupted by search for correct form of expression (2.5) Uses topic related vocabulary without problems (2.5) Responds to questions using varied and descriptive vocabulary and language structures (2.5)	Pronunciation is understandable, but there are slight errors (2.25) Speech is connected but frequently disrupted by search for correct form of expression (2.25) Uses some topic related vocabulary sufficient to communicate ideas (2.25) Responds to questions using simple but accurate vocabulary and language structures (2.25	Pronunciation is understandable most of the time, marked native accent and many errors (2) Speaks with simple sentences, sometimes not connected, but is understood (2) Uses basic vocabulary to communicate ideas (2) Partly responds to simple questions, with limited vocabulary and language structures (2)	Pronunciation makes language very difficult to understand (1) Uses one-word/two-word utterances (1) Unable to communicate ideas due to lack of vocabulary (1) Uses isolated words or sentence fragments to respond to questions (1)
5	Respuesta a los cuestiona-mientos de los evaluadores	Clara y satisfactoria	Clara y parcialmente satisfactoria	Clara e insuficiente	Confusa e insuficiente
5	Autorización de tesina en tiempo y forma	Presenta en tiempo y forma	Presenta en tiempo y forma con la mayoría de requerimientos solicitados	Presenta en tiempo y con algunas limitantes de los requerimientos solicitados.	Presenta fuera de tiempo y con los mínimos requerimientos solicitados.

Dr. Said Polanco Martagón	X	
ASESOR INSTITUCIONAL	EVALUADOR	EVALUADOR DE INGLÉS

REGISTRO DE EVALUACIÓN DE EXPOSICIÓN DE ESTADÍA

Siendo las X horas del día X, el alumno **Ángel Iván Cabrera Rojas**, del programa académico **Ingeniería en Tecnologías de la Información**, con matricula **2230343**, presentó la exposición de la estadía realizada durante el cuatrimestre **x**, en la **Universidad Politécnica de Victoria**, con el proyecto titulado **Implementación de fluentes de estado multivaluados en MDP-ProbLog mediante disyunciones anotadas.**.

Una vez concluido el proceso de evaluación, y con base a la rúbrica establecida para éste propósito, se determina que la calificación de la estadía es _____.

_____
Dr. Said Polanco Martagón
ASESOR INSTITUCIONAL

_____
X
EVALUADOR

_____

EVALUADOR DE INGLÉS

Agradecimientos

Resumen

La Programación Lógica Probabilística (PLP) es un paradigma que combina la capacidad de representación de la lógica de primer orden con la teoría de la probabilidad para el manejo de la incertidumbre. Este paradigma permite modelar una amplia variedad de dominios que abarcan situaciones de la vida real en donde existen interacciones complejas entre conjuntos de elementos heterogéneos y distintos niveles de incertidumbre. Dentro de este contexto, destaca el framework MDP-ProbLog, el cual utiliza las capacidades del lenguaje ProbLog para la especificación y resolución de Procesos de Decisión de Markov (MDPs) de horizonte infinito utilizando una representación factorizada, lógica y relacional.

Sin embargo, la arquitectura base de este sistema presenta una limitación estructural: la representación de los fluentes de estado se restringe exclusivamente a variables de Bernoulli independientes (booleanas). Debido a esto, para modelar variables que por naturaleza son multivaluadas, el usuario se ve obligado a utilizar técnicas para descomponerlas en múltiples predicados binarios. En el caso de MDP-ProbLog, esta restricción resulta en un proceso de modelado tedioso, propenso a inconsistencias semánticas y poco natural.

La presente tesina reporta el desarrollo de una extensión al núcleo del framework MDP-ProbLog para solucionar esta problemática. El trabajo abarca la revisión de la arquitectura del código fuente, el planteamiento y diseño de la extensión, la programación, la ejecución de pruebas y su respectiva documentación. Mediante modificaciones a los módulos de gestión de estados y del motor de inferencia, se integró el soporte para fluentes de estado multivaluados utilizando el mecanismo nativo de las Disyunciones Anotadas (Annotated Disjunctions). Como resultado de este desarrollo, el modelado de MDPs que requieren variables con dominios multivaluados se convierte en un proceso semánticamente íntegro, mucho más directo y natural para el usuario.

Palabras clave: Programación Lógica Probabilística, MDP-ProbLog, Disyunciones Anotadas, Fluentes Multivaluados, Espacio de Estados.

Summary

Probabilistic Logic Programming (PLP) is a paradigm that combines the representational power of first-order logic with probability theory to handle uncertainty. This paradigm allows for the modeling of a wide variety of domains encompassing real-world situations where complex interactions exist among sets of heterogeneous elements and varying levels of uncertainty. Within this context, the MDP-ProbLog framework stands out, leveraging the capabilities of the ProbLog language for the specification and resolution of infinite-horizon Markov Decision Processes (MDPs) using a factored, logical, and relational representation.

However, the base architecture of this system presents a structural limitation: the representation of state fluents is restricted exclusively to independent Bernoulli (Boolean) variables. Due to this, to model variables that are naturally multi-valued, the user is forced to use techniques to decompose them into multiple binary predicates. In the case of MDP-ProbLog, this restriction results in a modeling process that is tedious, prone to semantic inconsistencies, and unnatural.

This thesis reports the development of an extension to the core of the MDP-ProbLog framework to address this problem. The work encompasses the review of the source code architecture, the planning and design of the extension, programming, testing, and its respective documentation. Through modifications to the state management modules and the inference engine, support for multi-valued state fluents was integrated using the native mechanism of Annotated Disjunctions. As a result of this development, the modeling of MDPs requiring variables with multi-valued domains becomes a semantically sound, much more direct, and natural process for the user.

Keywords: Probabilistic Logic Programming, MDP-ProbLog, Annotated Disjunctions, Multi-valued Fluents, State Space.

Índice

Agradecimientos vii

Resumen viii

Summary ix

Índice x

1. Introducción 1
- 1.1. Antecedentes locales . . . 2
- 1.2. Definición del Problema . . . 3
- 1.3. Objetivos . . . 3
  - 1.3.1. Objetivo General . . . 3
  - 1.3.2. Objetivos Específicos . . . 3
- 1.4. Justificación . . . 4
- 1.5. Alcances y Limitaciones . . . 5
  - 1.5.1. Alcances . . . 5
  - 1.5.2. Limitaciones . . . 6

2. Estado del Arte 7
- 2.1. MDPs Factorizados y el Problema de la Representación de Estados . . . 7
- 2.2. Programación Lógica Probabilística para la Resolución de MDPs . . . 8
- 2.3. Disyunciones Anotadas y Compilación de Conocimiento . . . 9
- 2.4. Análisis Comparativo . . . 10
- 2.5. Conclusión del Estado del Arte . . . 11

3. Marco Teórico 12
- 3.1. Programación Lógica . . . 12
- 3.2. Programación Lógica Probabilística . . . 13
  - 3.2.1. Semántica de distribución . . . 13
  - 3.2.2. Relevancia para MDP-ProbLog . . . 15
- 3.3. Conteo de Modelos Ponderados . . . 16
  - 3.3.1. El lenguaje ProbLog . . . 17
- 3.4. Conteo de Modelos Ponderados . . . 18
- 3.5. Procesos de Decisión de Markov . . . 19
  - 3.5.1. Algoritmo de Iteración de Valor . . . 19
- 3.6. MDPs Factorizados . . . 20
  - 3.6.1. Fluentes de estado . . . 20
- 3.7. El Framework MDP-ProbLog . . . 22
- 3.8. Disyunciones Anotadas . . . 22
  - 3.8.1. Definición formal de LPADs . . . 22
  - 3.8.2. Semántica: selecciones e instancias . . . 23
  - 3.8.3. Exclusión mutua nativa . . . 23
  - 3.8.4. Interpretación causal . . . 24

3.9. Compilación de Conocimiento 24
3.9.1. Mapa de compilación de conocimiento 25
3.9.2. Impacto de la representación en el tamaño del circuito 25

4. Diseño e Implementación **26**
4.1. Análisis de la arquitectura original 26
4.1.1. Arquitectura base y flujo de ejecución 26
4.1.2. Limitaciones del diseño original en la gestión de fluentes 26
4.1.3. Identificación de puntos de extensión 27
4.2. Diseño de la extensión 28
4.2.1. FluentSchema: Estructura factorizada del espacio de estados 28
4.2.2. FluentClassifier: Pipeline de clasificación automática 29
4.2.3. Codificación *mixed-radix* del espacio de estados 30
4.2.4. Nueva sintaxis para fluentes multivaluados 31
4.3. Implementación 32
4.3.1. Modificaciones al adaptador del motor (`engine.py`) 32
4.3.2. Modificaciones al orquestador central (`mdp.py`) 33
4.3.3. Adaptaciones al algoritmo de resolución (`value_iteration.py`) . . . 33
4.3.4. Simulador y herramientas de depuración (`debugger.py`) 34
4.4. Retrocompatibilidad 34

Índice de figuras **36**

Índice de cuadros **37**

Índice de algoritmos **38**

Referencias **39**

1. Introducción

Dentro del ámbito de la Inteligencia Artificial, la Programación Lógica Probabilística se ha consolidado como uno de los paradigmas fundacionales el cual combina la expresividad de la lógica de primer orden con el manejo formal de la incertidumbre mediante la teoría de la probabilidad [1]. Dentro de los desarrollos apegados a este paradigma se encuentra *ProbLog*, un lenguaje que extiende Prolog mediante la introducción de hechos probabilísticos y cuya semántica se basa en la distribución sobre los mundos posibles [2]. ProbLog permite calcular la probabilidad de que una consulta lógica sea verdadera dado un conjunto de evidencias, utilizando técnicas de inferencia como el Conteo de Modelos Ponderados (Weighted Model Counting), transformando el problema de inferencia en una evaluación eficiente sobre circuitos lógicos compilados [3].

Paralelamente, en el campo de la toma de decisiones secuenciales, los Procesos de Decisión de Markov se han establecido como el estándar matemático para modelar agentes que actúan en entornos estocásticos. Formalizados rigurosamente a mediados del siglo XX con los trabajos de Richard Bellman sobre programación dinámica [4], los MDPs proporcionan un marco formal para optimizar una función de utilidad a lo largo del tiempo. Sin embargo, su representación clásica mediante matrices de estados atómicos presenta serias limitaciones de escalabilidad. A medida que aumenta el número de variables que describen el estado del sistema, el tamaño del espacio de estados crece de manera exponencial, un fenómeno acuñado por Bellman como la "maldición de la dimensionalidad" [4]. Esta explosión combinatoria hace que los métodos de resolución clásicos sean computacionalmente intratables para problemas del mundo real con múltiples variables interrelacionadas [5].

Integrando la expresividad lógica de ProbLog y la capacidad de decisión de los MDPs, Bueno et al. propusieron el framework **MDP-ProbLog** [6]. Esta herramienta integra los conceptos de estados, acciones y recompensas dentro de la sintaxis de la programación lógica, permitiendo representar MDPs de horizonte infinito de manera relacional y compacta. Al utilizar una representación factorizada, MDP-ProbLog mitiga parcialmente la problemática de la dimensionalidad al descomponer el estado global en un conjunto de características o fluentes. Actualmente, este programa es la base tecnológica del equipo de investigación liderado por el Dr. Héctor Hugo Avilés-Arriaga, quienes lo utilizan para el desarrollo de sistemas de navegación y toma de decisiones en vehículos autónomos.

1.1. Antecedentes locales

Actualmente, MDP-ProbLog constituye la base tecnológica del equipo de investigación liderado por el Dr. Héctor Hugo Avilés-Arriaga, quienes lo utilizan para el desarrollo de sistemas de navegación y toma de decisiones en vehículos autónomos. Sin embargo, el framework conserva desde su publicación una restricción que limita la naturalidad del modelado. Todos los fluentes de estado se definen exclusivamente como variables de Bernoulli independientes, lo que obliga al usuario a representar variables multivaluadas mediante múltiples predicados booleanos. Para modelar la posición de un vehículo en una vía de tres carriles, por ejemplo, se requieren tres fluentes binarios separados en lugar de una única variable con tres valores. Esta codificación forzada produce un proceso de modelado tedioso, propenso a inconsistencias semánticas, y genera un espacio de estados artificialmente inflado.

El presente trabajo aborda el diseño e implementación de fluentes de estado multivaluados en el núcleo de MDP-ProbLog mediante el uso de Disyunciones Anotadas (ADs) [7], con el objetivo de extender las capacidades de representación del sistema, garantizando la integridad semántica de los modelos y sentando las bases para analizar el impacto computacional de esta expresividad.

1.2. Definición del Problema

La arquitectura actual de MDP-ProbLog carece de un mecanismo nativo para representar y evaluar variables de estado que posean más de dos valores mutuamente excluyentes. Esta restricción a fluentes booleanos independientes genera una desconexión fundamental entre la naturaleza multivaluada de los fenómenos físicos del mundo real y su representación computacional.

Esta limitación delega en el usuario la responsabilidad de mantener la consistencia semántica mediante la escritura exhaustiva de reglas lógicas adicionales. La ausencia de soporte para fluentes multivaluados resulta en un proceso de modelado tedioso y poco natural, aumentando la propensión a errores humanos en la definición de las políticas de decisión y generando un espacio de estados artificialmente inflado con combinaciones semánticamente imposibles.

1.3. Objetivos

1.3.1. Objetivo General

Diseñar e integrar un mecanismo de representación de fluentes de estado multivaluados en el núcleo del framework MDP-ProbLog utilizando Disyunciones Anotadas (Annotated Disjunctions), con el fin de proveer una sintaxis de modelado más natural, asegurar la consistencia semántica en los procesos de decisión estocásticos y evaluar el impacto computacional derivado de esta nueva expresividad.

1.3.2. Objetivos Específicos

- Analizar la arquitectura interna actual de MDP-ProbLog, identificando los módulos críticos encargados de la gestión del espacio de estados y la inferencia probabilística para determinar los puntos de extensión.
- Desarrollar una estructura de datos y adaptar la lógica de instanciación (*grounding*) que permita procesar fluentes multivaluados frente a los fluentes binarios tradicionales.
- Implementar el soporte para Disyunciones Anotadas en la definición de las transiciones de estado, garantizando matemáticamente la exclusión mutua nativa de los valores de una variable sin necesidad de reglas de restricción externas.
- Evaluar el comportamiento de la extensión propuesta, contrastando la simplificación cualitativa del proceso de modelado con las implicaciones cuantitativas generadas en el tamaño de los modelos compilados y los tiempos de inferencia.

1.4. Justificación

El trabajo abarca un paso necesario para la maduración de la herramienta. Desde una perspectiva práctica y operativa, la integración de fluentes multivaluados impacta directamente la manera en la que se modelan y describen los MDPs. Al automatizar la exclusión mutua mediante Disyunciones Anotadas, se elimina la necesidad de crear reglas manuales restrictivas, reduciendo drásticamente la carga cognitiva del modelador, acortando los tiempos de desarrollo y mitigando la propensión a errores de diseño.

Para dimensionar el problema: una variable con k valores posibles requiere, bajo la codificación booleana actual, k predicados independientes que generan 2^k combinaciones, de las cuales solo k son semánticamente válidas. Por ejemplo, un semáforo de tres colores produce $2^3 = 8$ estados de los cuales 5 son imposibles por definicion; un fluente con 5 valores genera $2^5 = 32$ combinaciones en donde únicamente 5 son válidas. Con la extensión propuesta, la representación se reduce exactamente a los k estados legítimos.

[Diagrama o figura en donde se muestre la diferencia entre la representación actual y la propuesta, destacando la reducción del espacio de estados y la simplificación del modelado.]

1.5. Alcances y Limitaciones

1.5.1. Alcances

El proyecto abarca el diseño, la implementación y la validación de una extensión al núcleo del framework MDP-ProbLog que incorpora soporte para fluentes de estado multivaluados. El desarrollo comprende los siguientes elementos:

1. **Representación factorizada del espacio de estados.** Se diseñaron las estructuras de datos necesarias para que el espacio de estados admita factores con más de dos valores posibles, preservando la capacidad de iteración y la indexación eficiente sobre el conjunto completo de estados.

2. **Clasificación automática de fluentes.** Se implementó un mecanismo que determina automáticamente si un fluente declarado por el usuario es booleano o multivaluado, a partir del análisis de las Disyunciones Anotadas presentes en el programa. El usuario también puede especificar el tipo de forma explícita.

3. **Integración con el motor de inferencia.** Se adaptó la interfaz entre MDP-ProbLog y el motor de inferencia de ProbLog para soportar la inyección y el manejo de Disyunciones Anotadas como representación nativa de los fluentes multivaluados.

4. **Adaptación del algoritmo de resolución.** Se reestructuró el flujo de evaluación de transiciones y el algoritmo de Iteración de Valor para operar sobre transiciones agrupadas por factor, de modo que cada variable de estado se evalúa de acuerdo con su cardinalidad real.

5. **Herramientas de inspección.** Se desarrollaron utilidades para exportar las matrices de transición, las tablas de recompensas y el historial de convergencia, facilitando la verificación y el análisis de los modelos resueltos.

6. **Retrocompatibilidad.** La extensión preserva el funcionamiento de los modelos booleanos existentes sin requerir modificaciones por parte del usuario.

La validación funcional se realiza sobre dominios de referencia basados en grids, específicamente la grid de Mitchell y la grid de Russell, que constituyen problemas canónicos en la literatura de MDPs con soluciones óptimas conocidas. En cada dominio se contrastan dos formulaciones del mismo problema, una con codificación booleana manual y otra con fluentes multivaluados, evaluando la simplificación del proceso de modelado, la corrección de la política resultante y el comportamiento del solver en términos de convergencia y tamaño del espacio de estados.

1.5.2. Limitaciones

El desarrollo se circunscribe a MDPs de horizonte infinito con espacios de estados discretos y completamente observables. Las siguientes restricciones delimitan el alcance técnico de la extensión:

1. **Enumeración explícita del espacio de estados.** El algoritmo de Iteración de Valor recorre todos los pares estado-acción en cada iteración. La extensión modifica la forma en que se representan y se evalúan los estados, pero no altera la estrategia de recorrido. El costo por iteración permanece proporcional al producto del número total de estados por el número de acciones disponibles. Esta restricción no es introducida por el presente trabajo, sino heredada del diseño original de MDP-ProbLog, y constituye la frontera principal de escalabilidad del sistema. Dominios con un número elevado de variables de estado o con variables de cardinalidad alta producen espacios de estados cuya enumeración completa se vuelve computacionalmente costosa.

2. **Capa de representación exclusivamente.** El trabajo no modifica los algoritmos de inferencia exacta ni los compiladores de circuitos lógicos utilizados internamente por ProbLog. El efecto de las Disyunciones Anotadas sobre el tamaño de los circuitos compilados es un resultado emergente del compilador subyacente, no una optimización introducida por esta extensión.

3. **Convención de declaración de fluentes.** Todos los valores posibles de un fluente multivaluado deben declararse bajo el mismo nombre de predicado. Esta convención permite que el sistema identifique automáticamente los grupos mutuamente excluyentes, pero impide representar un fluente cuyos valores se distribuyan entre predicados distintos.

4. **Independencia entre variables de estado.** Las reglas de transición del programa deben respetar independencia entre las variables de estado. El sistema evalúa la probabilidad de transición de cada variable por separado y combina los resultados asumiendo que los factores son estocásticamente independientes. Si las reglas del usuario introducen dependencias entre variables que violan este supuesto, el sistema no lo detecta ni emite advertencias.

5. **Dominios de evaluación.** Las pruebas se ejecutan sobre dominios de grid discretos de dimensión reducida, seleccionados por la disponibilidad de soluciones de referencia en la literatura. Los resultados no se extrapolan a dominios de gran escala ni a entornos físicos en tiempo real.

Una vez delimitado el problema, los objetivos y los alcances de este trabajo, el siguiente capítulo analiza los antecedentes científicos que sustentan la propuesta, revisando los enfoques existentes para la representación factorizada de MDPs y el papel de la Programación Lógica Probabilística en la toma de decisiones secuenciales.

2. Estado del Arte

El presente capítulo revisa la literatura científica relacionada con la sustentación técnica de esta tesina: la representación factorizada de MDPs y el problema de las variables de estado, el uso de la Programación Lógica Probabilística como lenguaje de especificación para la toma de decisiones secuenciales, y los mecanismos de exclusión mutua y compilación de conocimiento que permiten la inferencia eficiente. El análisis de estos trabajos permite trazar la trayectoria que conduce al vacío de investigación que la presente tesina aborda: la ausencia de soporte nativo para fluentes de estado multivaluados en frameworks de resolución exacta de MDPs basados en PLP.

2.1. MDPs Factorizados y el Problema de la Representación de Estados

En la formulación clásica de un MDP, el espacio de estados se trata como un conjunto plano donde cada estado es una entidad atómica e indivisible [5]. Esta representación se vuelve poco práctica conforme aumenta el número de variables descritas: si un dominio involucra n variables binarias, el número total de estados crece como 2^n, lo que Bellman denominó la *maldición de la dimensionalidad* [4].

Para mitigar este crecimiento exponencial, Boutilier, Dearden y Goldszmidt propusieron la representación factorizada de MDPs, donde las transiciones se codifican mediante Redes Bayesianas Dinámicas (DBNs) y las funciones de valor se manipulan como árboles de decisión [8]. En esta representación, cada variable de estado posee su propia tabla de probabilidad condicional, y los algoritmos de programación dinámica operan sobre agregaciones de estados en lugar de iterar sobre cada estado individual. Su algoritmo *Structured Policy Iteration* (SPI) demostró que era posible resolver MDPs sin enumerar explícitamente el espacio de estados, siempre que la estructura del problema exhibiera independencias condicionales aprovechables.

En la misma línea, Hoey *et al.* desarrollaron SPUDD, un algoritmo de iteración de valor que reemplaza los árboles de decisión por Diagramas de Decisión Algebraicos (ADDs) [9]. Los ADDs, al ser grafos dirigidos acíclicos en lugar de árboles, permiten compartir subgrafos isomorfos, logrando reducciones de hasta 30 veces en el número de nodos respecto a las representaciones arbóreas. SPUDD demostró la resolución exacta de MDPs con hasta 63 millones de estados.

Tanto SPI como SPUDD comparten una restricción, ya que operan exclusivamente sobre variables booleanas. Cuando el dominio contiene variables con más de dos valores, el modelador debe descomponer cada variable multivaluada en múltiples variables binarias. Hoey *et al.* reconocen explícitamente esta limitación al señalar que variables ternarias deben expandirse en dos binarias, lo cual agranda el espacio de estados por un factor de 4/3 por cada variable expandida e introduce estados semánticamente imposibles [9]. Este problema de la binarización forzada es precisamente la limitación que la presente tesina aborda en el contexto de MDP-ProbLog.

En el ámbito de los lenguajes de descripción para planificación, RDDL (*Relational Dynamic Influence Diagram Language*) [10] sí contempla fluentes multivaluados de manera nativa, incluyendo variables enumeradas, enteras y continuas. No obstante, RDDL está fundamentado en redes bayesianas dinámicas proposicionales, lo que impide la representación de dependencias cíclicas que surgen naturalmente en dominios relacionales. MDP-ProbLog, en contraste, hereda de la programación lógica la capacidad de expresar este tipo de relaciones, pero carece del soporte multivaluado que RDDL ofrece.

2.2. Programación Lógica Probabilística para la Resolución de MDPs

La Programación Lógica Probabilística (PLP) combina la capacidad de representación relacional de la lógica de primer orden con el manejo formal de la incertidumbre mediante la teoría de la probabilidad. De Raedt y Kimmig establecieron los fundamentos unificadores de este paradigma, demostrando cómo la *semántica de distribución* permite definir una distribución de probabilidad sobre los mundos posibles de un programa lógico [11]. Bajo esta semántica, cada hecho probabilístico actúa como una variable aleatoria independiente cuya realización determina un programa lógico determinista; la probabilidad de una consulta es entonces la suma de las probabilidades de todos los mundos en los que dicha consulta es verdadera.

ProbLog, el lenguaje que implementa esta semántica, transforma el problema de inferencia probabilística en un problema de Conteo de Modelos Ponderados (WMC) sobre fórmulas booleanas [3, 1]. Esto permite aprovechar técnicas de compilación de conocimiento para lograr inferencia exacta en tiempo polinomial respecto al tamaño del circuito compilado.

La extensión de ProbLog hacia la toma de decisiones se materializó primero en DTProbLog [12], que añadió al lenguaje la capacidad de modelar problemas de decisión episódicos: dado un conjunto de variables de decisión y atributos de utilidad, el sistema encuentra la estrategia que maximiza la utilidad esperada. Sin embargo, DTProbLog está diseñado para problemas de un solo paso y no soporta la resolución de MDPs de horizonte infinito, donde es necesario optimizar una política completa a lo largo del tiempo.

Como derivado de este trabajo, Bueno *et al.* propusieron MDP-ProbLog, el cual extiende la sintaxis de ProbLog con predicados reservados para declarar fluentes de estado, acciones y utilidades, y emplea programas de distribución de transición de dos pasos temporales para representar la dinámica del MDP [6]. El *solver* implementa iteración de valor sobre el circuito compilado mediante WMC, demostrando formalmente la equivalencia entre ejecutar el programa probabilístico y realizar un *backup* de Bellman. Una ventaja distintiva de MDP-ProbLog frente a lenguajes como RDDL es su capacidad de representar dependencias cíclicas entre predicados aterrizados — como la influencia mutua entre individuos en un dominio de marketing viral — gracias a la resolución SLD de Prolog [6].

No obstante, la arquitectura de MDP-ProbLog hereda una restricción significativa: todos los fluentes de estado se definen como variables de Bernoulli independientes. Para representar

un fluente con k valores posibles, el usuario debe declarar k predicados booleanos y escribir manualmente las reglas de restricción que garanticen que exactamente uno esté activo en cada estado. Esta codificación manual es tediosa, propensa a errores y genera un espacio de estados artificialmente inflado con $2^k - k$ combinaciones semánticamente imposibles.

2.3. Disyunciones Anotadas y Compilación de Conocimiento

El mecanismo que esta tesina propone para resolver la restricción binaria de MDP-ProbLog son las Disyunciones Anotadas (ADs). Vennekens, Verbaeten y Bruynooghe formalizaron los *Logic Programs with Annotated Disjunctions* (LPADs), donde cada cláusula tiene la forma $(h_1 : \alpha_1) \vee \ldots \vee (h_n : \alpha_n) \leftarrow$ cuerpo, con $\sum \alpha_i = 1$ [7]. La semántica se define mediante *selecciones*: para cada cláusula, se elige exactamente un disyunto, produciendo un programa normal cuyo modelo bien fundado determina la interpretación. La probabilidad de una interpretación es la suma de las probabilidades de todas las selecciones que conducen a ella, asumiendo independencia entre las elecciones de distintas cláusulas.

Lo relevante de este formalismo para el presente trabajo es doble. Primero, las ADs garantizan *por construcción* que exactamente un disyunto es verdadero por cada cláusula, lo cual codifica la exclusión mutua de manera nativa sin necesidad de reglas adicionales. Segundo, las ADs admiten una interpretación causal natural: el cuerpo de la cláusula representa una causa, y los disyuntos en la cabeza son sus posibles efectos mutuamente excluyentes, cada uno con su probabilidad asociada [13]. Esta interpretación mapea directamente al concepto de transición estocástica en un MDP: dado un estado y una acción (causa), el siguiente valor de un fluente es uno de varios resultados posibles (efectos).

El trabajo de Vennekens *et al.* demostró además la equivalencia expresiva entre los LPADs y el *Independent Choice Logic* (ICL) de Poole [14] para programas acíclicos. Ambos formalismos comparten la idea de que la incertidumbre se modela mediante elecciones independientes, pero los LPADs ofrecen una sintaxis más natural para representar efectos indeterminados de acciones, ya que permiten colocar las probabilidades directamente en la cabeza de las reglas.

Para que la inferencia sobre programas con ADs sea computacionalmente tratable, ProbLog emplea un *pipeline* de compilación de conocimiento. El programa aterrizado se transforma primero en una fórmula proposicional ponderada, y posteriormente se compila en un circuito lógico. Darwiche y Marquis establecieron la taxonomía de los lenguajes de compilación, identificando que estructuras como las d-DNNFs (*Deterministic Decomposable Negation Normal Forms*) permiten inferencia en tiempo lineal respecto al tamaño del circuito [15]. Más recientemente, Darwiche introdujo los Diagramas de Decisión Sentencial (SDDs), una representación canónica que generaliza los BDDs y es estrictamente más sucinta [16]. ProbLog2 adopta los SDDs como su estructura de compilación principal [17], y es sobre estos circuitos donde MDP-ProbLog ejecuta el Conteo de Modelos Ponderados para calcular las probabilidades de transición.

2.4. Análisis Comparativo

La Tabla 1 presenta una comparativa de los principales enfoques revisados en este capítulo. Se analizan diez trabajos representativos agrupados por familia metodológica. Para cada uno se evalúan cuatro dimensiones: el tipo de variables de estado que soporta, el mecanismo de inferencia empleado, la capacidad del lenguaje para expresar relaciones entre entidades, y la forma en que se garantiza la exclusión mutua entre los valores de una misma variable.

Tabla 1: Análisis comparativo del estado del arte. Se evalúan cuatro dimensiones para cada trabajo: el tipo de variables de estado soportadas, el mecanismo de inferencia, la capacidad relacional del lenguaje, y la garantía de exclusión mutua entre valores de una misma variable.

Trabajo	**Variables de estado**	**Inferencia**	**Expresividad relacional**	**Exclusión mutua**
MDPs factorizados proposicionales				
Boutilier *et al.* [8]	Booleanas	Exacta (árboles)	Proposicional (DBNs)	Manual
Boutilier, Dean y Hanks [18]	Booleanas	Exacta (prog. dinámica)	Proposicional	Manual
Hoey *et al.* [9]	Booleanas (binarización)	Exacta (ADDs)	Proposicional (DBNs)	Manual
Lenguajes de planificación				
Sanner [10]	Multival., continuas	Exacta / Aprox.	Relacional (DBNs)	Nativa por tipo
Programación Lógica Probabilística				
De Raedt y Kimmig [11]	Booleanas	Exacta (WMC)	Relacional (ProbLog)	No aplica
Fierens *et al.* [3]	Booleanas	Exacta (WMC)	Relacional (ProbLog)	No aplica
Nitti *et al.* [19]	Discretas, continuas	Aprox. (muestreo)	Relacional (DDC)	Implícita
Riguzzi y Swift [20]	Booleanas	Exacta (tabling)	Relacional	No aplica
Exclusión mutua y semántica causal				
Vennekens *et al.* [13]	Multivaluadas (ADs)	Exacta	Relacional	Nativa
Poole [14]	Booleanas (ICL)	Exacta	Relacional	Nativa (elecciones)
MDP-ProbLog				
Bueno *et al.* [6]	Booleanas	Exacta (WMC/SDDs)	Relacional (ProbLog)	Manual
Propuesta	**Multivaluadas**	**Exacta (WMC/SDDs)**	**Relacional (ProbLog)**	**Nativa (ADs)**

Como se observa en la tabla, ninguno de los enfoques existentes integra simultáneamente las cuatro capacidades que la propuesta de esta tesina reúne: variables de estado multivaluadas con exclusión mutua nativa, inferencia exacta mediante WMC sobre circuitos compilados, y expresividad relacional completa heredada de la programación lógica.

2.5. Conclusión del Estado del Arte

La revisión de la literatura permite identificar tres hallazgos principales. En primer lugar, la representación factorizada de MDPs ha demostrado ser una estrategia efectiva para mitigar la maldición de la dimensionalidad, permitiendo resolver problemas con millones de estados sin enumeración explícita [8, 9]. Sin embargo, los enfoques proposicionales más consolidados operan exclusivamente sobre variables booleanas, y cuando el dominio requiere variables multivaluadas, la binarización forzada introduce estados semánticamente imposibles y aumenta artificialmente el tamaño de los circuitos compilados.

En segundo lugar, la Programación Lógica Probabilística ofrece ventajas significativas sobre los formalismos proposicionales al permitir representar relaciones simétricas, transitivas y cíclicas entre las variables del dominio [6, 11]. El framework MDP-ProbLog materializó esta ventaja al combinar la expresividad de ProbLog con la resolución exacta de MDPs de horizonte infinito mediante WMC [3]. No obstante, su arquitectura restringe los fluentes de estado a variables de Bernoulli independientes, lo que limita la naturalidad del proceso de modelado y delega en el usuario la responsabilidad de codificar manualmente las restricciones de exclusión mutua.

En tercer lugar, las Disyunciones Anotadas constituyen un mecanismo formal que garantiza la exclusión mutua de eventos probabilísticos a nivel semántico [13, 14]. Este constructo ya forma parte del lenguaje ProbLog y es utilizado internamente por su motor de inferencia [17], pero su integración en la capa de representación de fluentes de estado dentro de MDP-ProbLog no ha sido explorada ni implementada.

En consecuencia, la presente tesina se posiciona en la intersección de estos tres ejes al proponer la extensión de MDP-ProbLog para soportar fluentes de estado multivaluados mediante Disyunciones Anotadas. Esta integración busca preservar las garantías de inferencia exacta del sistema, eliminar la necesidad de binarización manual, y habilitar una representación del espacio de estados que sea tanto más compacta como semánticamente íntegra. El siguiente capítulo establece los fundamentos formales — desde la programación lógica hasta la compilación de conocimiento — que el lector necesitará para comprender el diseño de la extensión propuesta.

3. Marco Teórico

En el capítulo anterior se estableció que MDP-ProbLog carece de soporte nativo para variables de estado multivaluadas, y que las Disyunciones Anotadas constituyen un mecanismo candidato para resolver esta limitación. El presente capítulo examina los antecedentes científicos que sustentan esta propuesta desde tres ejes: la representación factorizada de MDPs y el problema de la binarización forzada de variables, el uso de la Programación Lógica Probabilística como lenguaje de especificación para la toma de decisiones secuenciales, y los mecanismos de exclusión mutua que permiten la inferencia eficiente. El análisis de estos trabajos permite identificar el vacío de investigación que la presente tesina aborda.

3.1. Programación Lógica

Dado que MDP-ProbLog está construido sobre el lenguaje Prolog, es necesario establecer los fundamentos de la programación lógica que sustentan tanto la representación del conocimiento como el proceso de inferencia del sistema. La programación lógica es un paradigma de computación donde los programas se expresan como conjuntos de sentencias lógicas y la ejecución consiste en realizar inferencias sobre dichas sentencias.

El vocabulario de un programa lógico se construye a partir de *constantes* (valores concretos como `rojo`, `a`), *variables* (denotadas con mayúscula inicial, como `X`) y *functores* (nombres de predicados con aridad asociada). Un *término* es una constante, variable o functor aplicado a términos; un *átomo* es un predicado aplicado a sus argumentos (e.g., `color(carro, rojo)`); y un *literal* es un átomo o su negación. A partir de estos elementos se construyen las cláusulas que componen un programa lógico.

Un programa lógico se compone de *cláusulas*, que pueden ser de tres tipos:

- **Hechos**: Afirmaciones incondicionales. Declaran que algo es verdadero sin depender de ninguna condición. Por ejemplo, `padre(tom, bob).` afirma que Tom es padre de Bob.

- **Reglas**: Implicaciones de la forma *cabeza* $\leftarrow$ *cuerpo*. Establecen que la cabeza es verdadera si todas las condiciones del cuerpo se satisfacen. En notación Prolog, esto se escribe como:

  ```
  abuelo(X, Z) :- padre(X, Y), padre(Y, Z).
  ```

  Esta regla establece que `X` es abuelo de `Z` si existe algún `Y` tal que `X` es padre de `Y` e `Y` es padre de `Z`.

- **Consultas**: Preguntas al sistema sobre qué es derivable del programa. Por ejemplo, `?- abuelo(tom, X).` pregunta quiénes son los nietos de Tom.

3.2. Programación Lógica Probabilística

Como se estableció en la sección anterior, la lógica de primer orden proporciona la estructura representacional base. Sin embargo, para modelar procesos estocásticos es imperativo cuantificar la incertidumbre. La Programación Lógica Probabilística (PLP) extiende la programación clásica con esta capacidad. Específicamente, la *semántica de distribución* define el marco probabilístico bajo el cual opera el motor de inferencia de MDP-ProbLog y constituye la base formal para el cálculo exacto de las probabilidades de transición entre estados.

3.2.1. Semántica de distribución

La base semántica de la PLP fue establecida por Sato mediante la *semántica de distribución* (*distribution semantics*) [2]. Su formulación parte de una observación elegante: si se asigna una distribución de probabilidad a los hechos de un programa lógico, las reglas del programa propagan esa incertidumbre de manera determinista, induciendo una distribución sobre el conjunto de todos los modelos mínimos posibles.

Formalmente, considérese un programa $DB = F \cup R$, donde $F = \{f_1, f_2, \ldots, f_k\}$ es un conjunto de hechos probabilísticos y R es un conjunto de reglas. Cada hecho f_i tiene asociada una probabilidad $\theta_i \in [0, 1]$ y se asume independencia estocástica entre todos los hechos. Una *realización total* (*total choice*) es una asignación de valores de verdad a todos los hechos de F: cada f_i es verdadero o falso. Dado que hay k hechos independientes, existen 2^k realizaciones posibles. La probabilidad de una realización L que incluye los hechos f_i como verdaderos y excluye los restantes es:

$$P(L \mid DB) = \prod_{f_i \in L} \theta_i \prod_{f_i \in F \setminus L} (1 - \theta_i) \tag{1}$$

Cada realización L determina un subconjunto de hechos verdaderos que, combinado con las reglas R, produce un programa lógico definido $L \cup R$ con un modelo mínimo único $M_{DB}(L)$. De esta forma, la distribución sobre realizaciones induce una distribución sobre modelos mínimos. Sato demostró que, bajo condiciones razonables (la *condición de soporte finito*), la masa de probabilidad se concentra exclusivamente en estos modelos derivados: $P_{DB}(\text{fix}(DB)) = 1$ [2].

La probabilidad de éxito (*success probability*) de una consulta q se define como la suma de las probabilidades de todas las realizaciones cuyo modelo mínimo satisface q:

$$P(q \mid DB) = \sum_{L : M_{DB}(L) \models q} P(L \mid DB) \tag{2}$$

Para ilustrar este mecanismo de manera concreta, considérese el siguiente programa mínimo con dos hechos probabilísticos y una regla:

```
0.3::llueve.
0.5::tiene_paraguas.
se_moja :- llueve, \+tiene_paraguas.
query(se_moja).
```

Código 1: Programa ProbLog mínimo para ilustrar la semántica de distribución.

Este programa define cuatro mundos posibles ($2^2 = 4$ realizaciones de los dos hechos). La Tabla 2 detalla cada una de estas realizaciones junto con su probabilidad calculada (Ecuación 1) y su modelo mínimo resultante.

Tabla 2: Mundos posibles y modelos mínimos inducidos por la semántica de distribución. Cada fila representa una realización total de los hechos probabilísticos independientes, su probabilidad conjunta y las consecuencias lógicas derivadas.

`llueve`	`tiene_paraguas`	P(mundo)	`se_moja`	Modelo mínimo
falso	falso	$0.7 \times 0.5 = 0.35$	falso	$\emptyset$
falso	verdadero	$0.7 \times 0.5 = 0.35$	falso	{`tiene_paraguas`}
verdadero	falso	$0.3 \times 0.5 = 0.15$	verdadero	{`llueve`, `se_moja`}
verdadero	verdadero	$0.3 \times 0.5 = 0.15$	falso	{`llueve`, `tiene_paraguas`}

La probabilidad de `se_moja` es la suma de las probabilidades de los mundos donde la consulta es verdadera: P(`se_moja`) $= 0.15$. Nótese cómo la regla lógica `se_moja :- llueve, \+tiene_paraguas.` propaga la incertidumbre de los hechos hacia la consulta de manera determinista dentro de cada mundo.

3.2.2. Relevancia para MDP-ProbLog

La semántica de distribución es el fundamento teórico sobre el cual opera todo el mecanismo de inferencia de MDP-ProbLog. Cuando el sistema evalúa una transición $P(x'_i = 1 \mid s, a)$, lo que hace internamente es:

1. *Fijar evidencia*: Los fluentes de estado actual y la acción se inyectan como hechos con probabilidad 1.0 (verdadero) o 0.0 (falso), colapsando los mundos posibles a aquellos consistentes con el estado y acción observados.

2. *Calcular la probabilidad de éxito*: El fluente de siguiente estado $x'_i(1)$ se trata como una consulta cuya probabilidad se computa sumando sobre los mundos posibles restantes (los que aún tienen incertidumbre, proveniente de los hechos probabilísticos en las reglas de transición).

De esta manera, la sustitución de evidencia en el circuito compilado es matemáticamente equivalente a calcular la probabilidad condicional $P(x'_i \mid s, a)$ bajo la semántica de distribución, sin necesidad de aplicar explícitamente la regla de Bayes. Esta equivalencia, demostrada formalmente por Bueno *et al.* [6], es lo que permite que cada evaluación del programa ProbLog corresponda a un *backup* de Bellman.

Aunque la semántica de distribución proporciona el marco teórico impecable para definir estas probabilidades, el cálculo directo sumando sobre mundos posibles crece exponencialmente con el número de hechos. Para resolver este desafío en la práctica, el problema se transforma utilizando técnicas de evaluación de fórmulas booleanas, concepto que se detalla a continuación.

3.3. Conteo de Modelos Ponderados

Para materializar la inferencia probabilística en tiempos computacionalmente tratables, ProbLog recurre al Conteo de Modelos Ponderados (WMC, por sus siglas en inglés). El WMC es la operación computacional central que MDP-ProbLog ejecuta de forma iterativa para calcular las probabilidades de transición. El cálculo directo de la Ecuación 2 requiere enumerar exponencialmente muchos mundos posibles, lo cual es computacionalmente intratable en general.

El procedimiento consta de tres pasos. Primero, se realiza el *grounding* relevante del programa respecto a la consulta, obteniendo un programa proposicional que contiene solo las cláusulas que pueden influir en el resultado. Segundo, este programa aterrizado se transforma en una fórmula booleana ponderada, donde cada variable proposicional tiene un peso asociado derivado de su probabilidad. Tercero, se calcula el conteo ponderado de los modelos de la fórmula:

$$\mathrm{WMC}(\Delta) = \sum_{\omega \models \Delta} \prod_{l \in \omega} w(l) \tag{3}$$

donde Δ es la fórmula booleana, ω recorre los modelos (asignaciones de verdad que satisfacen Δ), y $w(l)$ es el peso del literal l. Para un hecho probabilístico con probabilidad p, el peso del literal positivo es p y el del literal negativo es $1 - p$.

La probabilidad de la consulta se obtiene entonces como:

$$P(q) = \frac{\mathrm{WMC}(\Delta_q)}{\mathrm{WMC}(\Delta)} \tag{4}$$

donde Δ_q es la fórmula que incorpora la restricción de que q es verdadera. En la práctica, cuando no hay evidencia adicional, el denominador es 1 y la probabilidad se reduce al numerador.

Con las herramientas lógicas (para la representación del conocimiento) y el WMC (para la inferencia tratable) plenamente definidas, el marco de trabajo cuenta con los mecanismos necesarios para evaluar el estado estático de un sistema. El siguiente capítulo aborda cómo modelar agentes que interactúan y modifican su entorno de manera temporal mediante los Procesos de Decisión de Markov.

3.3.1. El lenguaje ProbLog

ProbLog es un lenguaje de programación lógica probabilística que implementa la semántica de distribución [1, 11]. Su extensión sintáctica respecto a Prolog es mínima: se introduce el concepto de *hecho probabilístico*, denotado por la anotación `p::hecho.`, que indica que el hecho es verdadero con probabilidad p e independiente de todos los demás hechos probabilísticos.

Para ilustrar la sintaxis, el siguiente programa — adaptado del dominio de marketing viral utilizado por Bueno *et al.* [6] y Van den Broeck *et al.* [12] — modela una red social simple donde las personas compran un producto ya sea por marketing directo o por influencia de sus contactos:

```
% Hechos probabilisticos (independientes)
0.3::compra_por_marketing(X) :- persona(X).
0.2::compra_por_influencia(X) :- persona(X).

% Hechos deterministas
persona(ana).  persona(bob).
confia(ana, bob).

% Reglas logicas
compra(X) :- compra_por_marketing(X).
compra(X) :- confia(X, Y), compra(Y),
             compra_por_influencia(X).

% Consulta
query(compra(ana)).
```

Código 2: Programa ProbLog para un dominio de marketing viral (adaptado de [6]).

En este programa, `compra_por_marketing(X)` es verdadero con probabilidad 0.3 de forma independiente para cada persona `X`. Las reglas determinan las consecuencias lógicas de estas realizaciones. ProbLog calcula $P(\texttt{compra(ana)})$ sumando sobre todos los mundos posibles en los que `compra(ana)` es derivable.

Un aspecto fundamental es que los hechos probabilísticos son estocásticamente independientes entre sí. Esto no es una limitación expresiva, ya que las dependencias entre variables se codifican mediante las reglas lógicas del programa [11].

3.4. Conteo de Modelos Ponderados

El Conteo de Modelos Ponderados es la operación computacional central que MDP-ProbLog ejecuta en cada iteración del algoritmo de Bellman para calcular las probabilidades de transición. El cálculo directo de la Ecuación **??** requiere enumerar exponencialmente muchos mundos posibles, lo cual es computacionalmente intratable en general.

El procedimiento consta de tres pasos. Primero, se realiza el *grounding* relevante del programa respecto a la consulta, obteniendo un programa proposicional que contiene solo las cláusulas que pueden influir en el resultado. Segundo, este programa aterrizado se transforma en una fórmula booleana ponderada, donde cada variable proposicional tiene un peso asociado derivado de su probabilidad. Tercero, se calcula el conteo ponderado de los modelos de la fórmula:

$$\mathrm{WMC}(\Delta) = \sum_{\omega \models \Delta} \prod_{l \in \omega} w(l) \tag{5}$$

donde Δ es la fórmula booleana, ω recorre los modelos (asignaciones de verdad que satisfacen Δ), y $w(l)$ es el peso del literal l. Para un hecho probabilístico con probabilidad p, el peso del literal positivo es p y el del literal negativo es $1 - p$.

La probabilidad de la consulta se obtiene entonces como:

$$P(q) = \frac{\mathrm{WMC}(\Delta_q)}{\mathrm{WMC}(\Delta)} \tag{6}$$

donde Δ_q es la fórmula que incorpora la restricción de que q es verdadera. En la práctica, cuando no hay evidencia adicional, el denominador es 1 y la probabilidad se reduce al numerador.

3.5. Procesos de Decisión de Markov

Los Procesos de Decisión de Markov (MDPs) formalizan el problema de decisión secuencial estocástica que la arquitectura de MDP-ProbLog está diseñada para resolver. Hasta este punto, se han descrito exclusivamente las herramientas lógicas y probabilísticas para evaluar el estado estático de un sistema. Sin embargo, para modelar agentes que interactúan, toman decisiones y modifican su entorno a lo largo del tiempo, es necesario incorporar este paradigma matemático.

En cada paso de tiempo, un agente observa el estado actual del sistema, ejecuta una acción, recibe una recompensa y transita a un nuevo estado según una distribución de probabilidad que depende únicamente del estado actual y la acción ejecutada [5].

3.5.1. Algoritmo de Iteración de Valor

La *Iteración de Valor* (*Value Iteration*) es un algoritmo de programación dinámica que calcula V^* mediante aproximaciones sucesivas. Partiendo de una función de valor inicial arbitraria $V^{(0)}$, se construye la secuencia:

$$V^{(k+1)}(s) = \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V^{(k)}(s') \right] \tag{7}$$

La secuencia $\{V^{(k)}\}$ converge a V^* bajo la norma supremo. Un criterio de convergencia comúnmente utilizado es detenerse cuando el residuo máximo satisface:

$$\max_{s \in \mathcal{S}} \left| V^{(k+1)}(s) - V^{(k)}(s) \right| \leq \frac{2\epsilon(1-\gamma)}{\gamma} \tag{8}$$

lo cual garantiza que la política *greedy* resultante es ϵ-óptima [5, 21].

Como se observa en la Ecuación 7, el cálculo requiere iterar sobre cada estado $s \in \mathcal{S}$ del sistema. En dominios complejos, representar este espacio como un conjunto plano e indivisible resulta computacionalmente inviable debido a la explosión combinatoria. Para mitigar este problema, se recurre a la factorización del espacio de estados, concepto que se detalla en la siguiente sección y que constituye el entorno estructural donde la distinción entre variables booleanas y multivaluadas adquiere su relevancia práctica.

3.6. MDPs Factorizados

En la formulación clásica, el espacio de estados $\mathcal{S}$ se trata como un conjunto plano de $|\mathcal{S}|$ elementos. Sin embargo, en la mayoría de los dominios prácticos, el estado del sistema se describe naturalmente mediante un conjunto de variables $\mathbf{x} = (x_1, x_2, \ldots, x_n)$, donde cada variable x_i toma valores en un dominio finito $\mathrm{Dom}(x_i)$. El espacio de estados resultante es el producto cartesiano $\mathcal{S} = \mathrm{Dom}(x_1) \times \mathrm{Dom}(x_2) \times \cdots \times \mathrm{Dom}(x_n)$, cuyo tamaño crece exponencialmente con n.

La factorización del espacio de estados es la estrategia central que permite a MDP-ProbLog operar sin enumerar explícitamente todas las combinaciones posibles, y es el contexto en el que la distinción entre fluentes booleanos y multivaluados adquiere relevancia práctica. En la formulación clásica, el espacio de estados $\mathcal{S}$ se trata como un conjunto plano de $|\mathcal{S}|$ elementos.

En un MDP factorizado, la función de transición se descompone aprovechando las independencias condicionales entre las variables de estado. Bajo el supuesto de que el valor futuro de cada variable x_i' depende solo de un subconjunto de las variables actuales y de la acción, la probabilidad de transición se factoriza como [8]:

$$P(s' \mid s, a) = \prod_{i=1}^{n} P(x_i' \mid \mathrm{Pa}(x_i'), a) \tag{9}$$

donde $\mathrm{Pa}(x_i') \subseteq \{x_1, \ldots, x_n\}$ denota los padres de x_i' en la Red Bayesiana Dinámica (DBN) que codifica las transiciones.

Esta factorización es la que permite a algoritmos como SPUDD [9] y a MDP-ProbLog [6] operar sin enumerar explícitamente todo el espacio de estados.

3.6.1. Fluentes de estado

En el contexto de MDP-ProbLog, las variables de estado se denominan *fluentes*, un término heredado del cálculo de situaciones que denota propiedades del mundo que pueden cambiar a lo largo del tiempo. Cada fluente se estampa temporalmente mediante un argumento adicional: el fluente `vivo` en el tiempo t se representa como `vivo(t)`, y su valor en el siguiente paso temporal como `vivo(t+1)`.

En la arquitectura original de MDP-ProbLog, cada fluente es una variable de Bernoulli que toma valores en $\{0, 1\}$. Para un dominio con n fluentes booleanos, el espacio de estados tiene 2^n estados posibles. La limitación de esta representación se manifiesta cuando el dominio contiene variables con más de dos valores mutuamente excluyentes. Por ejemplo, para representar la posición de un robot en una cuadrícula de tres celdas, se requieren tres fluentes booleanos (`en_celda1`, `en_celda2`, `en_celda3`), generando $2^3 = 8$ combinaciones de las cuales solo 3 son semánticamente válidas. Las 5 combinaciones restantes (el robot en dos celdas simultáneamente, o en ninguna) son estados imposibles que deben excluirse manualmente.

[Agregar una representación visual de el grid y la codificación de los fluentes para ilustrar este punto.] [Citar y mencionar si es necesario de donde vienen los modelos de grids]

3.7. El Framework MDP-ProbLog

MDP-ProbLog es un framework que combina la expresividad de ProbLog con la resolución de MDPs de horizonte infinito [6].

El solver de MDP-ProbLog implementa el Algoritmo 7 ejecutando los siguientes pasos en cada iteración: (1) para cada par estado-acción, construye un vector de evidencia que fija los valores de los fluentes actuales y la acción; (2) evalúa las probabilidades marginales de los fluentes en $t = 1$ mediante WMC sobre el circuito compilado; (3) calcula la recompensa inmediata y el valor esperado futuro; y (4) actualiza la función de valor. Este proceso se repite hasta alcanzar la convergencia.

La equivalencia formal entre ejecutar el programa probabilístico de MDP-ProbLog y realizar un *backup* de Bellman (Ecuación **??**) fue demostrada por Bueno *et al.* [6], garantizando que el solver produce políticas óptimas.

3.8. Disyunciones Anotadas

Las Disyunciones Anotadas constituyen el mecanismo formal central de la extensión propuesta en esta tesina. Su capacidad de codificar la exclusión mutua a nivel semántico — sin restricciones externas — es precisamente la propiedad que permite representar fluentes de estado multivaluados de forma nativa en MDP-ProbLog. Las Disyunciones Anotadas (ADs) son un constructo sintáctico y semántico de la PLP que permite representar elecciones probabilísticas entre múltiples alternativas mutuamente excluyentes de manera nativa [7].

3.8.1. Definición formal de LPADs

Un *Logic Program with Annotated Disjunctions* (LPAD) es un conjunto de reglas de la forma:

$$(h_1 : \alpha_1) \vee (h_2 : \alpha_2) \vee \cdots \vee (h_n : \alpha_n) \leftarrow b_1, \ldots, b_m \tag{10}$$

donde los h_i son átomos, los b_j son literales que conforman el cuerpo de la regla, y los $\alpha_i \in [0, 1]$ son probabilidades tales que $\sum_{i=1}^{n} \alpha_i = 1$. Cada regla expresa que, cuando el cuerpo es verdadero, exactamente uno de los disyuntos h_i se hace verdadero, con probabilidad α_i.

En la sintaxis de ProbLog, una AD se escribe de la siguiente manera:

```
% El semaforo cambia de color con cierta probabilidad
0.4::color(verde, 1) ; 0.25::color(amarillo, 1) ;
    0.35::color(rojo, 1) :- color(verde, 0).
```

Código 3: Ejemplo de Disyunción Anotada en ProbLog.

Esta cláusula establece que si el semáforo está en verde en el tiempo $t = 0$, entonces en $t = 1$ estará en verde con probabilidad 0.4, en amarillo con probabilidad 0.25, o en rojo con

probabilidad 0.35. La exclusión mutua (exactamente un color activo) está garantizada por construcción.

3.8.2. Semántica: selecciones e instancias

La semántica de un LPAD se define mediante el concepto de *selección* [7]. Una selección σ es una función que, para cada regla r del programa aterrizado, elige exactamente un par $(h_i : \alpha_i)$ de la cabeza de r. Cada selección produce una *instancia* P_σ: un programa lógico normal (sin disyunciones) donde solo el disyunto seleccionado permanece en la cabeza de cada regla.

La probabilidad de una selección se calcula asumiendo independencia entre las elecciones de distintas reglas:

$$C_\sigma = \prod_{r \in P} \sigma_{\text{prob}}(r) \tag{11}$$

donde $\sigma_{\text{prob}}(r)$ es la probabilidad del disyunto seleccionado por σ para la regla r.

La probabilidad de una interpretación I bajo el programa P es la suma de las probabilidades de todas las selecciones cuya instancia tiene a I como modelo bien fundado:

$$\pi^*_P(I) = \sum_{\sigma \in S(I)} C_\sigma \tag{12}$$

donde $S(I) = \{\sigma \mid \text{WFM}(P_\sigma) = I\}$ es el conjunto de selecciones que conducen a la interpretación I.

3.8.3. Exclusión mutua nativa

La propiedad más relevante de las ADs para este trabajo es que la exclusión mutua entre los disyuntos es una *consecuencia directa de la semántica*, no una restricción que deba imponerse externamente. Dado que cada selección elige exactamente un disyunto por regla, es imposible que dos disyuntos de la misma cláusula sean simultáneamente verdaderos en una misma instancia. Esto contrasta radicalmente con la codificación mediante hechos booleanos independientes, donde la exclusión mutua debe garantizarse mediante reglas lógicas adicionales escritas por el usuario.

Para ilustrar esta diferencia, considérese la representación de una variable con tres valores (por ejemplo, el color de un semáforo) bajo ambos enfoques:

```
% Tres fluentes booleanos independientes
state_fluent(verde).
state_fluent(amarillo).
state_fluent(rojo).
```

```
% Restricciones manuales de exclusion mutua
% (el usuario debe escribirlas explicitamente)
falso :- verde(0), amarillo(0).
falso :- verde(0), rojo(0).
falso :- amarillo(0), rojo(0).
falso :- not verde(0), not amarillo(0), not rojo(0).
```

Código 4: Codificación booleana (MDP-ProbLog original).

```
% Un unico fluente multivaluado con exclusion mutua nativa
state_fluent(semaforo(X)) :- color(X).

1/3::color(verde); 1/3::color(amarillo); 1/3::color(rojo).

% La transicion se expresa como una AD:
0.4::semaforo(verde, 1) ; 0.25::semaforo(amarillo, 1) ;
    0.35::semaforo(rojo, 1) :- semaforo(verde, 0).
% No se requieren restricciones adicionales.
```

Código 5: Codificación con Disyunciones Anotadas (extensión propuesta).

La codificación booleana genera $2^3 = 8$ combinaciones posibles, de las cuales solo 3 son válidas. La codificación con ADs genera exactamente 3 estados, sin combinaciones adicionales.

3.8.4. Interpretación causal

Vennekens, Denecker y Bruynooghe extendieron la semántica de los LPADs mediante *CP-Logic* (*Causal Probabilistic Logic*), formalizando una interpretación causal de las ADs [13]. Bajo esta interpretación, el cuerpo de cada regla representa una *causa* y los disyuntos en la cabeza son sus posibles *efectos*, cada uno con una probabilidad de ocurrencia asociada. Esta perspectiva causal mapea de manera natural al concepto de transición estocástica en un MDP: dado un estado y una acción (la causa), el siguiente valor de un fluente es uno de varios resultados mutuamente excluyentes (los efectos).

Es importante distinguir entre las probabilidades *causales* de las ADs y las probabilidades *condicionales* clásicas. Como señalan Vennekens *et al.*, si un mismo efecto puede ser producido por múltiples causas no excluyentes, la probabilidad causal de que una causa particular produzca el efecto no coincide con la probabilidad condicional del efecto dado la causa [7]. Las ADs modelan la primera, no la segunda.

3.9. Compilación de Conocimiento

La compilación de conocimiento determina la eficiencia computacional de la inferencia probabilística y es directamente afectada por la forma en que se representan las variables de estado. El cálculo del WMC (Ecuación 5) sobre una fórmula booleana arbitraria es un problema #P-completo en general [15], por lo que transformar el programa lógico en una estructura tratable

es un requisito ineludible para el framework.

3.9.1. Mapa de compilación de conocimiento

Darwiche y Marquis clasificaron los principales lenguajes de compilación según sus propiedades de sucintez y las operaciones que soportan en tiempo polinomial [15]. Para la inferencia probabilística mediante WMC, las representaciones más relevantes son:

- **d-DNNF** (*Deterministic Decomposable Negation Normal Form*): Un circuito booleano en forma normal de negación donde los nodos AND tienen hijos que no comparten variables (descomponibilidad) y los nodos OR tienen hijos mutuamente excluyentes (determinismo). Sobre esta estructura, el WMC se computa en un único recorrido ascendente (*bottom-up*) del circuito.

- **SDD** (*Sentential Decision Diagram*): Introducidos por Darwiche [16], los SDDs generalizan los Diagramas de Decisión Binarios Ordenados (OBDDs) al permitir decisiones sobre fórmulas en lugar de variables individuales. Los SDDs son canónicos (dado un *vtree*, la representación es única), soportan todas las operaciones booleanas en tiempo polinomial, y son estrictamente más sucintos que los OBDDs.

ProbLog utiliza los d-DNNF como su estructura de compilación principal [17].

3.9.2. Impacto de la representación en el tamaño del circuito

Un aspecto fundamental para este trabajo es que el tamaño del circuito compilado depende no solo de la complejidad intrínseca del problema, sino también de *cómo* se formula el programa lógico. Dos programas que describen el mismo MDP pero con diferentes codificaciones pueden producir circuitos de tamaños muy distintos.

En particular, codificar una variable de k valores mediante k hechos probabilísticos booleanos independientes introduce 2^k posibles asignaciones de verdad en el espacio de búsqueda del compilador, de las cuales solo k son semánticamente válidas. Las restricciones de exclusión mutua, aunque eliminan los modelos inválidos durante la inferencia, no necesariamente reducen el tamaño de la estructura compilada, ya que el compilador puede necesitar representar explícitamente la estructura lógica que descarta esas combinaciones.

En contraste, una Disyunción Anotada con k disyuntos codifica directamente las k alternativas válidas con la restricción $\sum \alpha_i = 1$ integrada en la semántica. ProbLog implementa las ADs internamente mediante nodos `choice` que garantizan estructuralmente la exclusión mutua, lo cual tiene el potencial de producir circuitos más compactos al evitar la representación de combinaciones imposibles desde el origen.

El impacto real de esta transición representacional sobre el tamaño de los circuitos generados y, por consiguiente, en los tiempos de inferencia computacional, constituye una de las hipótesis centrales de esta investigación. Este comportamiento se analizará y cuantificará empíricamente en el Capítulo de Experimentación y Resultados.

4. Diseño e Implementación

Una vez establecidos los fundamentos teóricos de la Programación Lógica Probabilística, los Procesos de Decisión de Markov factorizados y las Disyunciones Anotadas en el capítulo anterior, el presente capítulo aborda el diseño y la construcción de la extensión al núcleo de MDP-ProbLog. Específicamente, se analiza primero la arquitectura original del sistema para diagnosticar sus limitaciones estructurales en la representación del espacio de estados. A partir de esta evaluación, se detalla el diseño de las nuevas estructuras de datos y algoritmos de inferencia de tipos, culminando con la implementación de las modificaciones al motor de inferencia y al solucionador de Bellman. Finalmente, se demuestra cómo esta integración preserva la retrocompatibilidad con los modelos booleanos preexistentes.

4.1. Análisis de la arquitectura original

Para comprender la magnitud y el impacto de la extensión propuesta, es indispensable examinar cómo el framework MDP-ProbLog procesaba originalmente la información. El sistema fue concebido bajo un diseño modular que actúa como un puente orquestador entre un motor lógico subyacente y un solucionador matemático clásico [6].

4.1.1. Arquitectura base y flujo de ejecución

La arquitectura de software original se estructura fundamentalmente en cuatro componentes interdependientes: el adaptador al motor de inferencia (`engine.py`), el módulo de representación de estados (`fluent.py`), el orquestador central (`mdp.py`), y el solucionador numérico (`value_iteration.py`).

El ciclo de vida de un modelo comienza cuando el programa escrito en Prolog es analizado sintácticamente (*parsing*) e inyectado en una Base de Datos de Cláusulas (*ClauseDB*) gestionada por ProbLog. A través de la clase `MDP`, el sistema interroga esta base de datos buscando predicados reservados como `state_fluent/1` y `action/1` para descubrir los elementos que componen el dominio.

Posteriormente, el sistema realiza una inyección temporal: transforma las declaraciones abstractas agregando estampas de tiempo ($t = 0$ para el estado actual y $t = 1$ para el siguiente estado) e introduce "hechos probabilísticos *dummy*.^en la base de conocimientos. Tras ejecutar un *grounding* relevante (aterrizado lógico) y compilar las reglas en un circuito proposicional evaluable, el solucionador `ValueIteration` toma el control, enumerando las configuraciones posibles y solicitando al motor las probabilidades de transición marginales para ejecutar las actualizaciones de la ecuación de Bellman.

4.1.2. Limitaciones del diseño original en la gestión de fluentes

La principal motivación de esta investigación surge de una limitación estructural crítica en la capa de representación del framework original, la cual compromete tanto la integridad semántica de los modelos como la eficiencia del solucionador. Aunque MDP-ProbLog permite modelar variables de estado relacionales mediante el predicado reservado `state_fluent/1`,

su diseño interno asume estrictamente que cada instancia básica (*ground atom*) opera como una variable de Bernoulli estocásticamente independiente.

El módulo `fluent.py` original implementaba la clase `StateSpace` como un iterador combinatorio plano. Dado un conjunto de n fluentes booleanos, el sistema generaba mecánicamente el producto cartesiano completo, produciendo un espacio de estados de tamaño 2^n. Esta arquitectura resulta funcional para dominios puramente binarios, pero genera un desajuste profundo al modelar fenómenos del mundo real que poseen múltiples estados mutuamente excluyentes.

Para ilustrar esta deficiencia, considérese el modelo de una máquina industrial analizado durante la fase de diagnóstico del sistema. Si un objeto procesado puede encontrarse en uno de cuatro estados físicos mutuamente excluyentes — `sucio`, `limpio`, `pintado` o `expulsado` —, el framework obligaba al modelador a declarar cuatro fluentes booleanos independientes. Consecuentemente, el iterador `StateSpace` construía internamente $2^4 = 16$ configuraciones posibles. De estas, únicamente 4 configuraciones representan estados físicamente válidos (aquellas donde exactamente un fluente es verdadero), mientras que las 12 restantes corresponden a estados semánticamente imposibles o "fantasma", tales como un objeto que se encuentra simultáneamente `limpio` y `sucio`, o en ninguno de los estados a la vez.

Esta divergencia entre la semántica del dominio y la representación computacional genera dos impactos negativos directos. Primero, desde la perspectiva de la ingeniería del conocimiento, delega en el usuario la carga cognitiva de escribir exhaustivamente reglas lógicas adicionales para bloquear transiciones hacia los estados inválidos, aumentando la propensión a errores e inconsistencias. Segundo, desde la perspectiva de la eficiencia algorítmica, el algoritmo de Iteración de Valor implementado en `value_iteration.py` desperdicia recursos computacionales calculando respaldos de Bellman (*backups*) y valores esperados para estados inalcanzables.

La resolución de este cuello de botella exige abandonar la suposición de independencia booleana universal y adoptar un esquema capaz de agrupar lógicamente variables relacionadas. Identificar exactamente dónde y cómo intervenir la arquitectura heredada para integrar un manejo nativo de variables multivaluadas constituye el primer paso del diseño propuesto, el cual se detalla a continuación.

4.1.3. Identificación de puntos de extensión

El análisis de las limitaciones expuestas en la sección anterior revela que la solución no radica en alterar el motor de inferencia probabilística subyacente. La compilación a circuitos lógicos, como los Diagramas de Decisión Sentencial (SDDs), sigue siendo la estrategia computacional más eficiente para ejecutar el Conteo de Modelos Ponderados [17]. En consecuencia, la intervención arquitectónica no debe modificar el núcleo de ProbLog, sino localizarse en la capa intermedia o "puente semántico": el espacio donde las declaraciones de la lógica de primer orden se transforman en variables de estado evaluables para el solucionador de Markov.

A partir de este diagnóstico, se identificaron tres módulos críticos que requieren reingeniería profunda para soportar fluentes multivaluados nativos sin romper la arquitectura existente:

1. **Capa de representación (`fluent.py`):** Es imperativo abandonar el iterador combinatorio plano (`StateSpace`) basado en variables de Bernoulli. En su lugar, se requiere diseñar una nueva estructura de datos centralizada — un esquema factorizado — capaz de clasificar, agrupar variables semánticamente afines y habilitar una indexación matemática de bases mixtas (*mixed-radix*).

2. **Capa de inyección y *parsing* (`mdp.py` y `engine.py`):** La inicialización del MDP, que originalmente inyectaba hechos probabilísticos aislados ($0.5 :: f$) en el instante temporal $t = 0$, debe modificarse. El nuevo diseño debe ser capaz de inyectar Disyunciones Anotadas con una distribución de probabilidad uniforme para los fluentes multivaluados, garantizando la exclusión mutua desde la raíz estocástica del modelo antes de iniciar la compilación del circuito.

3. **Capa de resolución (`value_iteration.py`):** El cálculo iterativo del valor esperado en la actualización de Bellman debe reestructurarse. En lugar de evaluar combinaciones booleanas aisladas que generan estados imposibles, el algoritmo debe adaptarse para iterar recursivamente sobre los factores lógicamente agrupados, recorriendo únicamente las ramas estocásticas válidas de la transición.

Al acotar las modificaciones a estos tres puntos de extensión estratégicos, se preservan intactas las garantías matemáticas de inferencia exacta de ProbLog, al tiempo que se dota al framework MDP-ProbLog de una expresividad semántica superior y una mayor eficiencia en la exploración del espacio de estados.

Con los puntos de extensión claramente definidos, la siguiente sección aborda el diseño formal de la solución, comenzando por el desarrollo del esquema factorizado del espacio de estados y el algoritmo de clasificación responsable de inferir automáticamente la topología de los fluentes.

4.2. Diseño de la extensión

Habiendo acotado los puntos de intervención arquitectónica, esta sección detalla el diseño conceptual y lógico de la extensión propuesta. El objetivo principal es transitar de un modelo de variables booleanas planas e independientes hacia una topología estructurada que soporte fluentes de estado multivaluados de forma nativa. Para lograrlo, se diseñaron dos componentes centrales que actúan en la fase de preparación del MDP: una estructura de datos factorizada para encapsular el dominio (`FluentSchema`) y un motor de inferencia estructural para clasificar los fluentes (`FluentClassifier`).

4.2.1. FluentSchema: Estructura factorizada del espacio de estados

El primer paso en el rediseño consistió en reemplazar el almacenamiento de fluentes basado en listas planas por una estructura de datos centralizada denominada `FluentSchema`. Esta clase actúa como el mapa topológico del Proceso de Decisión de Markov, definiendo el espacio de estados no como una combinatoria binaria genérica, sino como el producto cartesiano de factores lógicamente independientes.

El diseño del esquema clasifica internamente cada factor del modelo en una de dos categorías probabilísticas:

- **ISF (*Independent State Fluents*):** Variables booleanas tradicionales que siguen una distribución de Bernoulli (base 2). Cada ISF constituye un factor independiente en el esquema.
- **ADS (*Annotated Disjunction States*):** Grupos de variables mutuamente excluyentes que siguen una distribución categórica o multinomial (base N). Un ADS agrupa múltiples opciones válidas bajo un único factor lógico.

Para gestionar los factores ADS de manera relacionalmente segura, el esquema implementa un mecanismo de *agrupamiento semántico* basado en firmas de predicados. Al instanciar el programa, el sistema agrupa las opciones evaluando su functor (el nombre del predicado) y sus argumentos base. Esto previene colisiones relacionales en dominios complejos; por ejemplo, garantiza que los valores `color(pieza_A, rojo)` y `color(pieza_A, azul)` se agrupen como alternativas excluyentes del mismo factor, separándolas estrictamente de `color(pieza_B, rojo)`.

Al encapsular esta topología, el `FluentSchema` provee las bases matemáticas para calcular las dimensiones reales del modelo y sentar las bases para la codificación numérica del espacio de estados, un proceso que depende inherentemente de la correcta clasificación previa de cada fluente.

4.2.2. FluentClassifier: Pipeline de clasificación automática

Para que el esquema factorizado se construya correctamente sin obligar al usuario a declarar exhaustiva y manualmente el tipo probabilístico de cada fluente, se diseñó e implementó el módulo `FluentClassifier`. Este componente ejecuta un proceso de *parsing* e inferencia de tipos en tres etapas: (1) validación estática de la sintaxis, (2) registro e inferencia de tipos lógicos, y (3) distribución y ensamblaje del esquema final.

El mérito algorítmico más significativo de este clasificador reside en su segunda etapa, donde se implementó un motor de inferencia por *Rastreo de Proveniencia* (*Provenance Tracing*). En lugar de depender de expresiones regulares o análisis de texto superficial para determinar si un fluente pertenece a un grupo multivaluado, el algoritmo navega directamente sobre el Árbol de Sintaxis Abstracta subyacente de la Base de Datos de Cláusulas (`ClauseDB`) compilada por ProbLog.

El proceso de inferencia estructural opera de la siguiente manera:

1. El clasificador identifica un predicado declarado mediante `state_fluent/1`.
2. Utilizando los índices *hash* internos del motor lógico (`db.find`), el algoritmo localiza el nodo de definición (`defnode`) exacto donde el fluente fue instanciado.
3. A partir de este puntero, el sistema desciende por la jerarquía lógica del programa

(`define` → `clause` → `conj` → `call`) buscando nodos estocásticos de tipo `choice` o `adc`.

4. Si la proveniencia del átomo se rastrea hasta el nodo interno de una Disyunción Anotada, el clasificador infiere automáticamente que la variable posee una naturaleza categórica y la etiqueta como un componente **ADS**. Por defecto, si el origen corresponde a un hecho simple, se clasifica como **ISF**.

Esta estrategia de introspección estructural dota al sistema de una alta robustez conceptual. El framework deduce la naturaleza probabilística del modelo basándose en la matemática de sus reglas lógicas subyacentes, permitiendo un modelado fluido y delegando la complejidad técnica al compilador.

Con los fluentes correctamente tipados y agrupados en el esquema semántico, el sistema dispone de la información necesaria para asignar un índice matemático único a cada estado del sistema, mecanismo que se detalla en la siguiente sección.

4.2.3. Codificación *mixed-radix* del espacio de estados

Con la topología de los fluentes correctamente inferida y agrupada por el `FluentClassifier`, el sistema se enfrenta al desafío matemático de iterar y enumerar este nuevo espacio de estados estructurado sin generar combinaciones inválidas. En la arquitectura original, la enumeración se lograva mediante un contador binario simple que asignaba un índice entero a cada combinación asumiendo una base 2 universal para todos los factores. Al introducir variables categóricas, esta suposición se rompe, haciendo indispensable un sistema de numeración capaz de manejar factores de cardinalidad arbitraria.

Para resolver este problema, la clase `StateSpace` se rediseñó para implementar una codificación de base mixta (*mixed-radix*). Bajo este esquema, el espacio de estados se concibe como el producto cartesiano de m factores independientes, donde cada factor k posee una base específica b_k correspondiente a su cardinalidad (por ejemplo, $b_k = 2$ para un fluente booleano y $b_k = N$ para un fluente multivaluado de N opciones).

El tamaño total del espacio de estados exacto y semánticamente válido se define entonces como el producto de todas las bases:

$$|\mathcal{S}| = \prod_{k=1}^{m} b_k \tag{13}$$

Para mapear biunívocamente cada estado posible a un índice entero secuencial $I \in [0, |\mathcal{S}|-1]$, el algoritmo calcula un "paso posicional.º *stride* (S_k) para cada factor. El *stride* representa el peso numérico de alterar el factor k, y se define como el producto de las bases de los factores precedentes:

$$S_k = \prod_{i=1}^{k-1} b_i \quad \text{con } S_1 = 1 \tag{14}$$

Dada una configuración específica del sistema donde cada factor k toma un valor local $v_k \in [0, b_k - 1]$, el índice global del estado se calcula mediante la sumatoria ponderada:

$$I = \sum_{k=1}^{m} (v_k \times S_k) \tag{15}$$

Esta codificación posicional ofrece una decodificación bidireccional eficiente en tiempo $\mathcal{O}(m)$. Elimina de raíz la generación de estados imposibles, ya que el espacio combinatorio iterado corresponde de manera exacta a las permutaciones físicamente válidas del modelo. Toda esta complejidad matemática operando en segundo plano (*backend*) tiene como propósito principal simplificar radicalmente la capa de modelado, como se expone en la siguiente subsección.

4.2.4. Nueva sintaxis para fluentes multivaluados

La justificación última de reestructurar la representación del estado y el motor de inferencia es proporcionar al modelador una interfaz declarativa más natural, segura y concisa. La extensión diseñada permite al usuario abstraerse por completo de las restricciones de exclusión mutua, delegando la responsabilidad de la coherencia semántica al propio compilador del framework.

En lugar de definir múltiples fluentes booleanos independientes y escribir exhaustivamente cláusulas lógicas restrictivas (como `falso :- opcion_A(0), opcion_B(0).`), el modelador ahora declara el fluente multivaluado de manera unificada y especifica sus transiciones directamente utilizando la sintaxis nativa de las Disyunciones Anotadas de ProbLog.

El siguiente código ilustra el paradigma de modelado resultante para el caso de estudio de la máquina industrial:

```
% Declaracion unificada del fluente de estado
state_fluent(estado_maquina(X)) :- objeto(X).

% Definicion del dominio multivaluado (ADS) en t=0
1/4::estado_maquina(sucio); 1/4::estado_maquina(limpio);
1/4::estado_maquina(pintado); 1/4::estado_maquina(expulsado).

% Transicion estocastica expresada elegantemente como una
   Disyuncion Anotada
0.8::estado_maquina(pintado, 1) ; 0.1::estado_maquina(limpio, 1)
   ;
    0.1::estado_maquina(sucio, 1) :- action(pintar(X)),
       estado_maquina(limpio, 0).
```

Código 6: Declaración y transición de un fluente multivaluado bajo la nueva sintaxis.

Bajo esta nueva sintaxis, el `FluentClassifier` infiere automáticamente que `estado_maquina` conforma un grupo **ADS** de base 4, basándose en la disyunción anotada que define sus valores

posibles. Simultáneamente, las reglas de transición codifican directamente la semántica causal estocástica, garantizando matemáticamente que $\sum \alpha_i = 1$ en cada salto temporal.

Con la abstracción del diseño completamente definida — desde su tipificación automática hasta su codificación matemática de bases mixtas y su sintaxis de usuario — el enfoque del capítulo se traslada ahora a la materialización técnica de estos conceptos. La siguiente sección detalla la implementación concreta de estas estructuras y las modificaciones algorítmicas realizadas sobre las capas de inyección y evaluación del sistema.

4.3. Implementación

Con el diseño conceptual de la extensión firmemente establecido, esta sección aborda la materialización de dichas estructuras abstractas a nivel de código fuente. La implementación se llevó a cabo mediante modificaciones quirúrgicas sobre los módulos principales del framework, garantizando la preservación de los algoritmos de inferencia exacta subyacentes. A continuación, se detallan las adaptaciones realizadas en el adaptador del motor (`engine.py`), el orquestador central (`mdp.py`) y el solucionador matemático (`value_iteration.py`).

4.3.1. Modificaciones al adaptador del motor (`engine.py`)

La clase `Engine` actúa como la interfaz directa entre la arquitectura de MDP-ProbLog y el núcleo de compilación de ProbLog. Para soportar la nueva semántica multivaluada, fue necesario extender la Interfaz de Programación de Aplicaciones (API) de este módulo con capacidades de inyección estocástica y de introspección avanzada.

La primera modificación significativa consistió en la implementación del método `add_annotated_disjunctio`
Originalmente, el sistema solo poseía la capacidad de inyectar hechos probabilísticos aislados mediante el método `add_fact`. El nuevo método permite inyectar dinámicamente Disyunciones Anotadas directamente en la Base de Datos de Cláusulas (`ClauseDB`). Al recibir un grupo de hechos mutuamente excluyentes y sus respectivas probabilidades, este componente construye internamente los nodos lógicos de tipo `choice` requeridos por el compilador, garantizando que el grafo proposicional resultante respete la exclusión mutua desde su concepción estructural.

La segunda incorporación fundamental fue el desarrollo del método de introspección `get_ads_metadata`. Este componente es el núcleo operativo del algoritmo de Rastreo de Proveniencia descrito en el diseño. El método recorre linealmente la tabla de instrucciones de la `ClauseDB` ya inicializada, identificando todos los nodos estocásticos de tipo `choice`. Para cada nodo detectado, extrae el término lógico subyacente y construye un índice invertido que mapea el valor en texto (*string*) hacia los identificadores lógicos de su grupo de exclusión (`group_id`). Este índice es posteriormente consumido por el `FluentClassifier` para etiquetar de manera inequívoca la naturaleza de cada variable de estado sin requerir declaraciones manuales del usuario.

4.3.2. Modificaciones al orquestador central (`mdp.py`)

La clase `MDP` funge como el puente orquestador que transforma el modelo lógico especificado por el usuario en un problema de optimización resoluble. Para integrar el esquema factorizado y las disyunciones anotadas en el flujo de trabajo, el método de inicialización interna `__prepare()` fue reescrito en su totalidad, transformándolo en un *pipeline* estructurado de cinco etapas secuenciales:

1. **Clasificación de fluentes:** El proceso inicia instanciando el `FluentClassifier`, el cual consume las reglas del programa y el índice de metadatos del motor para generar la estructura topológica definitiva en el `FluentSchema`.

2. **Inyección de estado actual ($t = 0$):** Utilizando el esquema generado, el sistema inyecta hechos *dummy* para representar el estado inicial. Los factores clasificados como booleanos (**ISF**) se inyectan con una probabilidad neutral de 0.5, mientras que los factores multivaluados (**ADS**) se inyectan utilizando el nuevo método `add_annotated_disjunction` con probabilidades uniformes de $1/N$ (siendo N la cardinalidad del factor).

3. **Inyección de acciones:** Siguiendo el mismo principio matemático de exclusión mutua nativa, el conjunto total de acciones declaradas en el dominio se inyecta en la `ClauseDB` como una única Disyunción Anotada con distribución uniforme.

4. **Aterrizado lógico (*Grounding* relevante):** El sistema construye una lista estricta de consultas (*queries*) combinando las utilidades, los fluentes de siguiente estado ($t = 1$), las acciones y los fluentes de estado actual ($t = 0$). La inclusión explícita de los fluentes en $t = 0$ constituye una modificación técnica crítica introducida en esta extensión: obliga al compilador de ProbLog a preservar los identificadores lógicos de estos átomos durante el proceso de optimización, lo cual es estrictamente necesario para su posterior localización al momento de inyectar evidencia.

5. **Compilación:** Finalmente, los fluentes de estado futuro y las funciones de utilidad se envían al compilador para generar el circuito evaluable (SDD o d-DNNF), mapeando los términos de la lógica de primer orden a los nodos del grafo para permitir inferencia en tiempo polinomial.

Esta reestructuración integral del flujo de preparación garantiza que el modelo probabilístico transferido al solucionador matemático sea semánticamente consistente con la realidad física del dominio modelado. Con el circuito lógico debidamente compilado, tipado y protegido contra estados inválidos, el siguiente y último eslabón de la implementación radica en adaptar el algoritmo de Iteración de Valor para que aproveche esta nueva topología factorizada.

4.3.3. Adaptaciones al algoritmo de resolución (`value_iteration.py`)

La adopción de una representación de estados de base mixta (*mixed-radix*) exige que el algoritmo de resolución sea capaz de navegar esta nueva topología de manera coherente. El corazón computacional del solucionador radica en el cálculo de la esperanza matemática del

valor futuro durante las actualizaciones síncronas de Bellman. En la implementación original, este proceso enumeraba exhaustivamente todas las combinaciones binarias posibles.

Para aprovechar la estructura semántica de los fluentes multivaluados, se rediseñó el método `__expected_value`. La nueva implementación ejecuta un recorrido recursivo sobre el árbol estocástico definido por las transiciones factorizadas. En cada nivel de recursión k, correspondiente a un factor específico del `FluentSchema`, el algoritmo:

1. Extrae las ramas activas (posibles valores futuros y sus probabilidades marginales) retornadas por el motor de inferencia.

2. Aplica un filtro de dispersión (*sparsity*), descartando cualquier rama cuya probabilidad sea inferior a 10^{-6}, lo cual reduce drásticamente el espacio de búsqueda en transiciones deterministas o cuasi-deterministas.

3. Calcula el índice global del estado sucesor acumulando el producto del índice local de la rama por el *stride* posicional del factor k.

4. Multiplica la probabilidad conjunta del camino estocástico y profundiza al siguiente factor.

Esta adaptación algorítmica garantiza que el operador de Bellman actualice los valores $V(s)$ iterando exclusivamente sobre transiciones físicamente posibles, eliminando el desperdicio computacional asociado a la evaluación de estados "fantasma".

4.3.4. Simulador y herramientas de depuración (`debugger.py`)

Dada la complejidad inherente a la Programación Lógica Probabilística y la abstracción introducida por la compilación de circuitos, dotar al framework de mecanismos de observabilidad resulta esencial para verificar la correctitud del modelo. Por ello, se desarrolló e integró el módulo `MDPDebugger`.

Esta herramienta permite realizar una inspección profunda o radiografía"de las estructuras internas del motor lógico. Entre sus capacidades destacan la exportación de la `ClauseDB` para auditar la compilación de las reglas, y el volcado (*dump*) de la topología construida por el `FluentSchema`. Más aún, el depurador incluye rutinas para reconstruir y exportar las matrices densas de transición $P(s' \mid s, a)$ y recompensas $R(s, a)$ a partir de las evaluaciones factorizadas, así como el historial completo de convergencia de la Iteración de Valor.

Estas utilidades no solo facilitaron la validación del algoritmo de inferencia de tipos durante el desarrollo, sino que proporcionan al modelador final los artefactos necesarios para analizar el comportamiento estocástico de su dominio.

4.4. Retrocompatibilidad

Un principio fundamental en el diseño de extensiones de *software* es la preservación del funcionamiento de los sistemas heredados. A pesar de la reestructuración profunda en la

gestión del espacio de estados y la inyección de hechos, la presente extensión fue diseñada para garantizar una retrocompatibilidad absoluta con los modelos booleanos existentes de MDP-ProbLog.

Esta garantía se sustenta en la lógica defensiva del `FluentClassifier`. Cuando el sistema procesa un programa Prolog clásico que define fluentes mediante variables independientes y restricciones manuales, el algoritmo de Rastreo de Proveniencia no detectará ninguna Disyunción Anotada en el árbol de sintaxis. En consecuencia, el esquema asignará por defecto la categoría **ISF** (base 2) a cada variable detectada.

Bajo esta clasificación, el cálculo de los *strides* posicionales degenera matemáticamente en la serie de potencias binarias clásica $(1, 2, 4, 8, \dots)$, y la inyección probabilística retrocede a la inyección de hechos aislados con probabilidad neutral $(0.5 :: f)$. De este modo, los archivos `.pl` heredados son procesados de manera idéntica a la arquitectura original, permitiendo al usuario adoptar la nueva sintaxis multivaluada de manera gradual.

Con la arquitectura, los algoritmos y la implementación completamente definidos, el sistema cuenta ahora con la capacidad técnica para modelar y resolver MDPs multivaluados de manera nativa. El siguiente capítulo abandona el plano del diseño para someter esta extensión a una evaluación empírica. Mediante la ejecución de casos de estudio, se cuantificará el impacto de esta reestructuración tanto en la simplificación del modelado como en los tiempos de compilación e inferencia del motor.

Índice de figuras

Índice de cuadros

1. Análisis comparativo del estado del arte. Se evalúan cuatro dimensiones para cada trabajo: el tipo de variables de estado soportadas, el mecanismo de inferencia, la capacidad relacional del lenguaje, y la garantía de exclusión mutua entre valores de una misma variable. 10
2. Mundos posibles y modelos mínimos inducidos por la semántica de distribución. Cada fila representa una realización total de los hechos probabilísticos independientes, su probabilidad conjunta y las consecuencias lógicas derivadas. 14

Índice de algoritmos

Referencias

[1] Luc De Raedt, Angelika Kimmig y Hannu Toivonen. "ProbLog: A probabilistic prolog and its application in link discovery". En: *IJCAI*. Vol. 7. 2007, págs. 2462-2467.

[2] Taisuke Sato. "A statistical learning method for logic programs with distribution semantics". En: *Proceedings of the 12th International Conference on Logic Programming (ICLP'95)*. Citeseer, 1995.

[3] Daan Fierens et al. "Inference and learning in probabilistic logic programs using weighted Boolean formulas". En: *Theory and Practice of Logic Programming* 15.3 (2015), págs. 358-401.

[4] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[5] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

[6] Thiago P Bueno et al. "Markov decision processes specified by probabilistic logic programming: Representation and solution". En: *2016 5th Brazilian Conference on Intelligent Systems (BRACIS)*. IEEE. 2016, págs. 337-342.

[7] Joost Vennekens, Sofie Verbaeten y Maurice Bruynooghe. "Logic programs with annotated disjunctions". En: *International Conference on Logic Programming*. Springer, 2004, págs. 431-445.

[8] Craig Boutilier, Richard Dearden y Moisés Goldszmidt. "Stochastic dynamic programming with factored representations". En: *Artificial Intelligence* 121.1-2 (2000), págs. 49-107.

[9] Jesse Hoey et al. "SPUDD: Stochastic Planning using Decision Diagrams". En: *Proceedings of the 15th Conference on Uncertainty in Artificial Intelligence (UAI-99)*. 1999, págs. 279-288.

[10] Scott Sanner. "Relational Dynamic Influence Diagram Language (RDDL): Language Description". Unpublished Manuscript, Australian National University. 2010.

[11] Luc De Raedt y Angelika Kimmig. "Probabilistic (logic) programming concepts". En: *Machine Learning* 100.1 (2015), págs. 5-47.

[12] Guy Van den Broeck et al. "DTProbLog: A Decision-Theoretic Probabilistic Prolog". En: *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence (AAAI-10)*. AAAI Press, 2010, págs. 1217-1222.

[13] Joost Vennekens, Marc Denecker y Maurice Bruynooghe. "CP-logic: A language of causal probabilistic events and its relation to logic programming". En: *Theory and Practice of Logic Programming* 9.3 (2009), págs. 245-308.

[14] David Poole. "The Independent Choice Logic for Modelling Multiple Agents under Uncertainty". En: *Artificial Intelligence* 94.1-2 (1997), págs. 7-56.

[15] Adnan Darwiche y Pierre Marquis. "A knowledge compilation map". En: *Journal of Artificial Intelligence Research* 17 (2002), págs. 229-264.

[16] Adnan Darwiche. "SDD: A New Canonical Representation of Propositional Knowledge Bases". En: *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI)*. 2011, págs. 819-826.

[17] Jonas Vlasselaer et al. "ProbLog2: Inference and learning in a probabilistic logic programming language". En: *Theory and Practice of Logic Programming* 16.5-6 (2016), págs. 673-689.

[18] Craig Boutilier, Thomas Dean y Steve Hanks. "Decision-theoretic planning: Structural assumptions and computational leverage". En: *Journal of Artificial Intelligence Research* 11 (1999), págs. 1-94.

[19] Davide Nitti, Vaishak Belle y Luc De Raedt. "Planning in discrete and continuous Markov decision processes by probabilistic programming". En: *Machine Learning* 104.2-3 (2016), págs. 327-342.

[20] Fabrizio Riguzzi y Terrance Swift. "Well-definedness and efficient inference for probabilistic logic programming under the distribution semantics". En: *Theory and Practice of Logic Programming* 13.2 (2013), págs. 279-302.

[21] Richard S Sutton y Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.